

Advanced Objects: Prototypes & Prototype Chain

Chapter 10-এ আমরা Object-এর basic থেকে advanced অনেক কিছু শিখেছি। এবার JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ **Object-related advanced concept**-গুলোর একটি শিখব:

Prototype → Prototype Chain → Constructor Function → `new` → `Object.create()` → Inheritance

এগুলো ভালোভাবে বুঝলে পরে **Class, OOP, Inheritance, React/Node.js**-এর অনেক concept বুঝতে সহজ হবে।

PART 01 — PROTOTYPES, CHAINS & PROPERTY LOOKUPS (11.1 – 11.14)

11.1 Prototype কী?

JavaScript-এর প্রতিটি সাধারণ object-এর পেছনে একটি **prototype object** থাকতে পারে।

সহজভাবে:

Prototype হলো এমন একটি object, যেখান থেকে অন্য object property ও method খুঁজে পেতে পারে।

💡 🏠 REAL-LIFE EXAMPLE

ধরো, একটি বিশ্ববিদ্যালয়ের অনেক Student আছে।

প্রত্যেক Student-এর নিজের:

TEXT

```
name  
id  
department
```

আছে।

কিন্তু সবাই একই ধরনের কিছু কাজ করতে পারে:

TEXT

```
study()  
attendClass()  
takeExam()
```

প্রত্যেক Student object-এর মধ্যে আলাদাভাবে `study()` লিখে না রেখে একটি common জায়গায় রাখা যায়।

সেই common জায়গাটা prototype-এর মতো।

11.2 Prototype দেখব কীভাবে?

JavaScript-এ:

11.2 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(Object.getPrototypeOf(user));
```

CONSOLE OUTPUT

```
{ ... }
```

এটি `user` object-এর prototype দেখাবে।

আরেকটি common syntax:

11.2 JAVASCRIPT

RUN

```
console.log(user.__proto__);
```

তবে `__proto__` legacy/accessor হিসেবে পরিচিত। শেখার সময় দেখতে পারো, কিন্তু code-এ prototype নিয়ে কাজ করার জন্য সাধারণত:

11.2 JAVASCRIPT

RUN

```
Object.getPrototypeOf()
```

এবং

11.2 JAVASCRIPT

RUN

```
Object.setPrototypeOf()
```

ব্যবহার করা ভালো।

11.3 Prototype Chain কী?

এটা খুব গুরুত্বপূর্ণ! 🔥

যখন JavaScript কোনো object-এর মধ্যে কোনো property বা method খুঁজে পায় না, তখন সে তার prototype-এর মধ্যে খোঁজে।

সেখানেও না পেলে prototype-এর prototype-এ খোঁজে।

এভাবে চলতে থাকে।

এটিই:

Prototype Chain

EXAMPLE

11.3 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(user.toString());
```

আমরা `user` object-এর মধ্যে নিজেরা `toString()` লিখিনি।

তবুও কাজ করছে।

কেন?

কারণ JavaScript roughly এভাবে খুঁজছে:

```
user  
↓  
Object.prototype  
↓  
null
```

`user` -এ `toString()` নেই।

তাই:

```
Object.prototype
```

এ গিয়ে `toString()` পেয়ে যায়।

11.4 Prototype Chain-এর Visual Idea

```
user  
|  
↓ prototype  
Object.prototype  
|  
↓ prototype  
null
```

এখানে:

11.4 JAVASCRIPT

RUN

```
Object.getPrototypeOf(user)
```

দিলে `Object.prototype` পাওয়া যাবে।

আর:

11.4 JAVASCRIPT

RUN

```
Object.getPrototypeOf(Object.prototype)
```

দিলে:

11.4 JAVASCRIPT

RUN

```
null
```

11.5

Object.prototype

JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ built-in prototype-এর একটি হলো:

11.5 JAVASCRIPT

RUN

`Object.prototype`

এখানে অনেক common method রয়েছে।

যেমন:

11.5 JAVASCRIPT

RUN

```
toString()
hasOwnProperty()
isPrototypeOf()
propertyIsEnumerable()
```

তাই অনেক object সরাসরি এগুলো ব্যবহার করতে পারে।

💡 EXAMPLE

11.5 JAVASCRIPT

RUN

```
const user = {
  name: "Ripon"
};

console.log(user.hasOwnProperty("name"));
```

CONSOLE OUTPUT

true

`user` object-এর নিজের property হলো:

TEXT

name

কিন্তু `hasOwnProperty()` object-এর নিজের property হিসেবে লিখিনি।

এটি prototype chain থেকে পাওয়া।

11.6 Property Lookup কীভাবে কাজ করে?

ধরো:

11.6 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(user.age);
```

JavaScript প্রথমে দেখবে:

TEXT

user.age

আছে?

না।

তারপর:

user → prototype

prototype-এ age আছে?

না।

তারপর prototype-এর prototype...

শেষে:

TEXT

null

এ পৌঁছাবে।

তখন result:

11.6 JAVASCRIPT

RUN

undefined

💡 EXAMPLE

11.6 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(user.age);
```

CONSOLE OUTPUT

undefined

11.7 নিজের Property বনাম Inherited Property

একটি property object-এর নিজের হতে পারে অথবা prototype থেকে inherited হতে পারে।

💡 EXAMPLE

11.7 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(user.name);  
console.log(user.toString);
```

এখানে:

TEXT

name

নিজের property।

কিন্তু:

TEXT

toString

prototype থেকে এসেছে।

11.8 hasOwnProperty()

কোনো property object-এর নিজের property কিনা দেখতে:

11.8 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(user.hasOwnProperty("name"));  
console.log(user.hasOwnProperty("toString"));
```

CONSOLE OUTPUT

true
false

কারণ:

name → own property
toString → inherited property

11.9 in Operator বনাম hasOwnProperty()

এটা খুব গুরুত্বপূর্ণ।

in

Prototype-এর property-ও খুঁজবে।

11.9 JAVASCRIPT

RUN

```
const user = {
  name: "Ripon"
};

console.log("name" in user);
console.log("toString" in user);
```

CONSOLE OUTPUT

```
true
true
```

কারণ `toString()` prototype থেকে পাওয়া যায়।

hasOwnProperty()

শুধু নিজের property check করবে।

11.9 JAVASCRIPT

RUN

```
console.log(user.hasOwnProperty("name"));
console.log(user.hasOwnProperty("toString"));
```

CONSOLE OUTPUT

```
true
false
```

মনে রাখো

Method	Own Property	Prototype Property
in	✓	✓
hasOwnProperty()	✓	✗

11.10 Object.getPrototypeOf()

কোনো object-এর prototype দেখতে:

11.10 JAVASCRIPT

RUN

```
const user = {
  name: "Ripon"
};

console.log(Object.getPrototypeOf(user));
```

সাধারণ object-এর ক্ষেত্রে সাধারণত:

`Object.prototype`

পাওয়া যাবে।

11.11 Object.setPrototypeOf()

একটি object-এর prototype পরিবর্তন করা যায়:

11.11 JAVASCRIPT

RUN

```
const person = {  
  greet() {  
    console.log("Hello!");  
  }  
};  
  
const user = {  
  name: "Ripon"  
};  
  
Object.setPrototypeOf(user, person);  
  
user.greet();
```

CONSOLE OUTPUT

Hello!

কারণ `user`-এর নিজের মধ্যে `greet()` নেই।

কিন্তু prototype:

`person`

হওয়ায় সে `greet()` পেয়েছে।

11.12 Prototype দিয়ে Inheritance

এখন আসল বিষয়।

ধরো:

11.12 JAVASCRIPT

RUN

```
const person = {  
  greet() {  
    console.log("Hello!");  
  }  
};  
  
const student = Object.create(person);  
  
student.name = "Ripon";  
  
student.greet();
```

CONSOLE OUTPUT

Hello!

এখানে:

```
student  
  ↓  
person  
  ↓  
Object.prototype  
  ↓  
null
```

এটাই prototype-based inheritance।

11.13

Object.create()

`Object.create()` দিয়ে নির্দিষ্ট prototype ব্যবহার করে নতুন object তৈরি করা যায়।

Syntax

11.13 JAVASCRIPT

RUN

```
Object.create(prototype);
```

EXAMPLE

11.13 JAVASCRIPT

RUN

```
const animal = {  
  eat() {  
    console.log("Animal is eating");  
  }  
};  
  
const dog = Object.create(animal);  
  
dog.name = "Tommy";  
  
console.log(dog.name);  
dog.eat();
```

CONSOLE OUTPUT

```
Tommy  
Animal is eating
```

`dog` -এর নিজের property:

TEXT

```
name
```

আর inherited method:

TEXT

```
eat()
```

11.14 Real-Life Example — University

11.14 JAVASCRIPT

RUN

```
const university = {
  attendClass() {
    console.log("Attending university class");
  }
};

const student = Object.create(university);

student.name = "Ripon";
student.department = "CSE";

console.log(student.name);
console.log(student.department);

student.attendClass();
```

CONSOLE OUTPUT

```
Ripon
CSE
Attending university class
```

এখানে:

```
student
  ↓
university
```

PART 02 — CONSTRUCTOR FUNCTIONS, 'NEW' KEYWORD & PROTOTYPE MEMORY (11.15 – 11.29)

11.15 Constructor Function কী?

Class আসার আগে JavaScript-এ object তৈরির জন্য খুব common technique ছিল:

Constructor Function

💡 EXAMPLE

11.15 JAVASCRIPT

RUN

```
function Student(name, department) {
  this.name = name;
  this.department = department;
}
```

এটি একটি constructor function হিসেবে ব্যবহার করা যাবে।

11.16 new Keyword

Constructor function-এর সাথে সাধারণত:

11.16 JAVASCRIPT

RUN

`new`

ব্যবহার করা হয়।

11.16 JAVASCRIPT

RUN

```
function Student(name, department) {  
  this.name = name;  
  this.department = department;  
}  
  
const student1 = new Student("Ripon", "CSE");  
  
console.log(student1);
```

```
Student {  
  name: "Ripon",  
  department: "CSE"  
}
```

11.17 new কী করে?

 new keyword-এর কাজ বোঝা খুব গুরুত্বপূর্ণ।

যখন লিখি:

11.17 JAVASCRIPT

RUN

```
const student1 = new Student("Ripon", "CSE");
```

Conceptually JavaScript roughly:

Step 1

নতুন empty object তৈরি করে।

11.17 JAVASCRIPT

RUN

```
{}
```

Step 2

নতুন object-এর prototype সেট করে:

```
Student.prototype
```

Step 3

`this` নতুন object-কে refer করে।

11.17 JAVASCRIPT

RUN

```
this.name = "Ripon";
```

Step 4

Constructor-এর মাধ্যমে object initialize করে।

Step 5

নতুন object return করে।

Conceptually:

```
new Student()  
  ↓  
new object  
  ↓  
prototype = Student.prototype  
  ↓  
this = new object  
  ↓  
return object
```

11.18 Constructor Function-এর Method কোথায় রাখা ভালো?

এভাবে করলে:

```
11.18 JAVASCRIPT RUN
function Student(name) {
  this.name = name;

  this.study = function() {
    console.log(this.name + " is studying");
  };
}
```

প্রতিটি student object-এর জন্য নতুন `study()` function তৈরি হবে।

যদি হাজার হাজার object থাকে, unnecessary duplication হতে পারে।

তার বদলে prototype ব্যবহার করা যায়।

11.19 Constructor.prototype

প্রতিটি normal constructor function-এর একটি:

```
11.19 JAVASCRIPT RUN
prototype
```

property থাকে।

Example:

```
11.19 JAVASCRIPT RUN
function Student(name) {
  this.name = name;
}

console.log(Student.prototype);
```

11.20 Prototype-এ Method যোগ করা

```
11.20 JAVASCRIPT RUN
function Student(name) {
  this.name = name;
}

Student.prototype.study = function() {
  console.log(this.name + " is studying");
};

const student1 = new Student("Ripon");
const student2 = new Student("Rahim");

student1.study();
student2.study();
```

CONSOLE OUTPUT

```
Ripon is studying
Rahim is studying
```

এখানে `study()` দুই object-এর মধ্যে duplicate হয়ে তৈরি হয়নি।

দুইজনই ব্যবহার করছে:

```
Student.prototype.study
```

11.21 Prototype Method-এর Memory Advantage

ধরো:

11.21 JAVASCRIPT

RUN

```
const student1 = new Student("Ripon");
const student2 = new Student("Rahim");
const student3 = new Student("Karim");
```

যদি method constructor-এর ভিতরে থাকে:

11.21 JAVASCRIPT

RUN

```
this.study = function() {}
```

তাহলে প্রতিটি instance-এর জন্য আলাদা function তৈরি হতে পারে।

কিন্তু prototype-এ:

11.21 JAVASCRIPT

RUN

```
Student.prototype.study = function() {}
```

একটি shared method ব্যবহার করা হয়।

Conceptually:

```
student1  ┌
student2 ┌─> Student.prototype.study()
student3 ┌
```

11.22 Constructor Function-এর Prototype Chain

ধরো:

11.22 JAVASCRIPT

RUN

```
function Student(name) {
  this.name = name;
}

Student.prototype.study = function() {
  console.log("Studying ... ");
};

const student = new Student("Ripon");
```

তাহলে:

```
student
  ↓
Student.prototype
  ↓
Object.prototype
  ↓
null
```

🔥 এই chain খুব গুরুত্বপূর্ণ।

11.23 `constructor` Property

`Student.prototype` -এর মধ্যে সাধারণত একটি:

```
11.23 JAVASCRIPT  
constructor
```

RUN

property থাকে।

```
11.23 JAVASCRIPT  
  
function Student(name) {  
  this.name = name;  
}  
  
console.log(Student.prototype.constructor === Student);
```

RUN

CONSOLE OUTPUT

true

অর্থাৎ:

```
Student.prototype.constructor
```

মূল constructor function-কে refer করে।

11.24 `instanceof`

কোনো object কোনো constructor-এর instance কিনা দেখতে:

```
11.24 JAVASCRIPT  
instanceof
```

RUN

ব্যবহার করা হয়।

💡 EXAMPLE

```
11.24 JAVASCRIPT  
  
function Student(name) {  
  this.name = name;  
}  
  
const student = new Student("Ripon");  
  
console.log(student instanceof Student);
```

RUN

CONSOLE OUTPUT

true

💡 আরেকটি EXAMPLE

```
11.24 JAVASCRIPT  
console.log(student instanceof Object);
```

RUN

CONSOLE OUTPUT

true

কারণ:

```
student  
↓  
Student.prototype  
↓  
Object.prototype
```

তাই `Object` -এর prototype chain-এর অংশ।

11.25 instanceof কীভাবে কাজ করে?

ধরো:

11.25 JAVASCRIPT

RUN

```
student instanceof Student
```

JavaScript দেখে:

```
Student.prototype
```

prototype chain-এর মধ্যে আছে কি না।

যদি থাকে:

TEXT

```
true
```

না থাকলে:

TEXT

```
false
```

11.26 Prototype Inheritance Example

11.26 JAVASCRIPT

RUN

```
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function() {
  console.log("Hello, I am " + this.name);
};

function Student(name, department) {
  Person.call(this, name);
  this.department = department;
}

Student.prototype = Object.create(Person.prototype);

Student.prototype.constructor = Student;

Student.prototype.study = function() {
  console.log(this.name + " is studying " + this.department);
};

const student = new Student("Ripon", "CSE");

student.sayHello();
student.study();
```

CONSOLE OUTPUT

```
Hello, I am Ripon
Ripon is studying CSE
```

এখানে:

```
student
↓
Student.prototype
↓
Person.prototype
↓
Object.prototype
↓
null
```

এটাই prototype-based inheritance-এর একটি traditional pattern।

11.27 `Person.call(this, name)` কী?

এই line:

```
11.27 JAVASCRIPT
```

RUN

```
Person.call(this, name);
```

`Person` constructor-কে current `this` context দিয়ে call করছে।

`Person` -এর মধ্যে:

```
11.27 JAVASCRIPT
```

RUN

```
this.name = name;
```

ছিল।

তাই Student-এর object-এর মধ্যেও:

```
11.27 JAVASCRIPT
```

RUN

```
name
```

set হচ্ছে।

11.28 `Object.create(Person.prototype)` কেন?

এই line:

```
11.28 JAVASCRIPT
```

RUN

```
Student.prototype = Object.create(Person.prototype);
```

এর মাধ্যমে বলা হচ্ছে:

Student-এর prototype হবে এমন একটি object, যার prototype হলো Person.prototype।

অর্থাৎ:

```
Student.prototype  
  ↓  
Person.prototype
```

11.29 কেন `constructor` আবার সেট করা হয়?

এই line:

```
11.29 JAVASCRIPT
```

RUN

```
Student.prototype.constructor = Student;
```

কারণ:

```
11.29 JAVASCRIPT
```

RUN

```
Student.prototype = Object.create(Person.prototype);
```

করার পর `constructor` reference বদলে যেতে পারে।

তাই explicitly:

```
11.29 JAVASCRIPT
```

RUN

```
Student.prototype.constructor = Student;
```

দেওয়া হয়।

11.30 Modern JavaScript-এ `class`

বর্তমানে JavaScript-এ সাধারণত:

11.30 JAVASCRIPT

RUN

`class`

ব্যবহার করা হয়।

কিন্তু মনে রাখবে:

JavaScript-এর `class` internally prototype-based system-এর উপর কাজ করে।

11.31 Class Example

11.31 JAVASCRIPT

RUN

```
class Student {
  constructor(name, department) {
    this.name = name;
    this.department = department;
  }

  study() {
    console.log(this.name + " is studying");
  }
}

const student = new Student("Ripon", "CSE");

student.study();
```

CONSOLE OUTPUT

Ripon is studying

11.32 Class Method কোথায় থাকে?

এই:

11.32 JAVASCRIPT

RUN

```
study() {
  console.log("Studying");
}
```

method সাধারণত প্রতিটি instance-এর নিজের property হিসেবে duplicate হয়ে থাকে না।

এটি:

`Student.prototype`

এর মাধ্যমে shared থাকে।

তাই:

11.32 JAVASCRIPT

RUN

```
console.log(
  Object.getPrototypeOf(student) === Student.prototype
);
```

CONSOLE OUTPUT

true

11.33 Class এবং Prototype-এর সম্পর্ক

11.33 JAVASCRIPT

RUN

```
class Student {  
  study() {  
    console.log("Studying");  
  }  
}
```

Conceptually:

```
Student  
  |  
  | prototype  
  v  
Student.prototype  
  |  
  | study()  
  v  
student instance
```

যখন:

11.33 JAVASCRIPT

RUN

```
student.study();
```

চালানো হয়, JavaScript prototype chain-এর মাধ্যমে `study()` খুঁজে পায়।

11.34 Inheritance with `extends`

Modern JavaScript-এ inheritance সহজভাবে:

11.34 JAVASCRIPT

RUN

`extends`

দিয়ে করা যায়।

11.34 JAVASCRIPT

RUN

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  
  sayHello() {  
    console.log("Hello, I am " + this.name);  
  }  
}  
  
class Student extends Person {  
  study() {  
    console.log(this.name + " is studying");  
  }  
}  
  
const student = new Student("Ripon");  
  
student.sayHello();  
student.study();
```

CONSOLE OUTPUT

```
Hello, I am Ripon  
Ripon is studying
```

11.35 এখানে Prototype Chain কী?

```
student
  ↓
Student.prototype
  ↓
Person.prototype
  ↓
Object.prototype
  ↓
null
```

তাই `student` :

11.35 JAVASCRIPT

RUN

```
student.study();
```

পায় `Student.prototype` থেকে।

আর:

11.35 JAVASCRIPT

RUN

```
student.sayHello();
```

পায় `Person.prototype` থেকে।

11.36

super

Parent class-এর constructor বা method ব্যবহার করতে:

11.36 JAVASCRIPT

RUN

super

ব্যবহার করা হয়।

💡 EXAMPLE

11.36 JAVASCRIPT

RUN

```
class Person {
  constructor(name) {
    this.name = name;
  }

  introduce() {
    console.log("My name is " + this.name);
  }
}

class Student extends Person {
  constructor(name, department) {
    super(name);
    this.department = department;
  }

  study() {
    console.log(this.name + " is studying " + this.department);
  }
}

const student = new Student("Ripon", "CSE");

student.introduce();
student.study();
```

CONSOLE OUTPUT

```
My name is Ripon
Ripon is studying CSE
```

11.37 `super()` কী করে?

Parent class:

11.37 JAVASCRIPT

RUN

Person

এর constructor:

11.37 JAVASCRIPT

RUN

```
constructor(name) {  
  this.name = name;  
}
```

কে call করছে:

11.37 JAVASCRIPT

RUN

```
super(name);
```

অর্থাত্:

```
Student  
  ↓  
super(name)  
  ↓  
Person constructor
```

11.38 Method Overriding

Child class parent-এর একই method নিজের মতো করে লিখতে পারে।

💡 EXAMPLE

11.38 JAVASCRIPT

RUN

```
class Animal {  
  speak() {  
    console.log("Animal makes a sound");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log("Dog barks");  
  }  
}  
  
const dog = new Dog();  
  
dog.speak();
```

CONSOLE OUTPUT

Dog barks

Child-এর method priority পায়।

11.39 `super.method()`

Parent-এর method-ও চাইলে call করা যায়।

```
11.39 JAVASCRIPT RUN
class Animal {
  speak() {
    console.log("Animal makes a sound");
  }
}

class Dog extends Animal {
  speak() {
    super.speak();
    console.log("Dog barks");
  }
}

const dog = new Dog();

dog.speak();
```

CONSOLE OUTPUT

```
Animal makes a sound
Dog barks
```

PART 04 — META-PROGRAMMING: DESCRIPTORS, DEFINEPROPERTY & GETTERS/SETTERS (11.40 – 11.54)

11.40 Prototype Pollution সম্পর্কে Basic ধারণা

এটি security-related গুরুত্বপূর্ণ concept।

যদি untrusted user input দিয়ে object prototype modify করার সুযোগ তৈরি হয়, application-এর অন্য object-ও প্রভাবিত হতে পারে।

Conceptually dangerous pattern:

```
11.40 JAVASCRIPT RUN
Object.prototype.isAdmin = true;
```

এখন:

```
11.40 JAVASCRIPT RUN
const user = {};

console.log(user.isAdmin);
```

CONSOLE OUTPUT

```
true
```

যদিও `user` -এর মধ্যে আমরা:

```
11.40 JAVASCRIPT RUN
isAdmin
```

লিখিনি।

কারণ এটি:

```
Object.prototype
```

থেকে এসেছে।

⚠️ বাস্তব application-এ এভাবে built-in prototypes modify করা উচিত নয়।

11.41

Object.create(null)

Prototype ছাড়া object তৈরি করা যায়:

11.41 JAVASCRIPT

RUN

```
const data = Object.create(null);

data.name = "Ripon";

console.log(data.name);
```

CONSOLE OUTPUT

Ripon

এখানে:

11.41 JAVASCRIPT

RUN

```
Object.getPrototypeOf(data)
```

এর output:

TEXT

null

অর্থাৎ prototype chain নেই।

11.42

কেন

Object.create(null)

ব্যবহার করা হয়?

বিশেষ কিছু dictionary/map-like data structure-এ এটি useful হতে পারে।

11.42 JAVASCRIPT

RUN

```
const dictionary = Object.create(null);

dictionary.name = "Ripon";
dictionary.role = "Developer";
```

এখানে inherited properties নেই।

তবে সাধারণ application data-এর জন্য সাধারণত:

11.42 JAVASCRIPT

RUN

{ }

ব্যবহার করাই সহজ।

11.43 Prototype বনাম Property

ধরো:

11.43 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};
```

এখানে:

TEXT

name

হলো নিজের property।

কিন্তু:

11.43 JAVASCRIPT

RUN

```
user.toString
```

prototype chain থেকে আসতে পারে।

Mental Model

Own Properties

name: "Ripon"

↓

Prototype

Object.prototype
toString()
hasOwnProperty()

↓

null

11.44 Object.getOwnPropertyNames()

Object-এর নিজের property names দেখতে:

11.44 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon",  
  age: 25  
};  
  
console.log(Object.getOwnPropertyNames(user));
```

CONSOLE OUTPUT

```
["name", "age"]
```

এটি prototype-এর properties দেখাবে না।

11.45 Object.getOwnPropertyDescriptors()

আরও বিস্তারিত property information দেখতে:

11.45 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(  
  Object.getOwnPropertyDescriptors(user)  
);
```

এতে property-এর:

TEXT

```
value  
writable  
enumerable  
configurable
```

ইত্যাদি information পাওয়া যায়।

11.46 Property Descriptor

Example:

11.46 JAVASCRIPT

RUN

```
const user = {  
  name: "Ripon"  
};  
  
console.log(  
  Object.getOwnPropertyDescriptor(user, "name")  
);
```

Conceptually output:

TEXT

```
{  
  value: "Ripon",  
  writable: true,  
  enumerable: true,  
  configurable: true  
}
```

এগুলোর অর্থ

Property	Meaning
value	value কী
writable	value পরিবর্তন করা যাবে কিনা
enumerable	loop/Object.keys-এ দেখা যাবে কিনা
configurable	property delete/descriptor change করা যাবে কিনা

11.47 Object.defineProperty()

Property-এর behavior control করা যায়।

11.47 JAVASCRIPT

RUN

```
const user = {};  
  
Object.defineProperty(user, "name", {  
  value: "Ripon",  
  writable: false  
});  
  
console.log(user.name);
```

CONSOLE OUTPUT

Ripon

এখন:

11.47 JAVASCRIPT

RUN

```
user.name = "Rahim";
```

করলেও সাধারণ strict-mode environment-এ error হতে পারে, আর non-strict mode-এ assignment silently fail করতে পারে।

11.48 Getter

Object property read করার সময় function চালানো যায়।

11.48 JAVASCRIPT

RUN

```
const user = {  
  firstName: "Shariar",  
  lastName: "Ripon",  
  
  get fullName() {  
    return this.firstName + " " + this.lastName;  
  }  
};  
  
console.log(user.fullName);
```

CONSOLE OUTPUT

Shariar Ripon

খেয়াল করো:

11.48 JAVASCRIPT

RUN

```
user.fullName
```

লিখেছি।

11.48 JAVASCRIPT

RUN

```
user.fullName()
```

লিখিনি।

কারণ এটি getter।

11.49 Setter

Property set করার সময় function চালানো যায়।

11.49 JAVASCRIPT

RUN

```
const user = {
  name: "Ripon",

  set username(value) {
    this.name = value;
  }
};

user.username = "Rahim";

console.log(user.name);
```

CONSOLE OUTPUT

Rahim

11.50 Getter + Setter Real-Life Example

11.50 JAVASCRIPT

RUN

```
const account = {
  _balance: 1000,

  get balance() {
    return this._balance;
  },

  set balance(amount) {
    if (amount ≥ 0) {
      this._balance = amount;
    }
  }
};

console.log(account.balance);

account.balance = 5000;

console.log(account.balance);
```

CONSOLE OUTPUT

1000
5000

এখানে setter-এর মাধ্যমে validation করা হয়েছে।

11.51 Prototype বনাম Class

বিষয়	Prototype	Class
JavaScript-এর মূল object model	✓	Prototype-এর উপর syntax
Syntax	তুলনামূলক complex	সহজ
Inheritance	<code>Object.create()</code> ইত্যাদি	<code>extends</code>
Constructor	Function	<code>constructor()</code>
Method	<code>.prototype</code>	class body
Beginner-friendly	কম	বেশি
Modern code	কম direct	বেশি ব্যবহৃত

🔥 মনে রাখবে:

JavaScript class হলো prototype-based inheritance-এর উপর তৈরি cleaner syntax।

11.52 একটি Complete Example

এবার সব একসাথে দেখি।

11.52 JAVASCRIPT

RUN

```
class Person {
  constructor(name) {
    this.name = name;
  }

  introduce() {
    console.log("My name is " + this.name);
  }
}

class Student extends Person {
  constructor(name, department, id) {
    super(name);

    this.department = department;
    this.id = id;
  }

  study() {
    console.log(
      this.name +
      " is studying " +
      this.department
    );
  }
}

const student = new Student(
  "Ripon",
  "CSE",
  "23105101019"
);

student.introduce();
student.study();

console.log(student instanceof Student);
console.log(student instanceof Person);
console.log(student instanceof Object);
```

CONSOLE OUTPUT

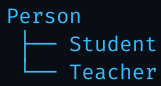
```
My name is Ripon
Ripon is studying CSE
true
true
true
```

Prototype Chain

```
student
↓
Student.prototype
↓
Person.prototype
↓
Object.prototype
↓
null
```

11.53 Real-Life Web Development Example

ধরো একটি university system-এ:



তাহলে:

11.53 JAVASCRIPT

RUN

```
class Person {
  constructor(name) {
    this.name = name;
  }

  login() {
    console.log(this.name + " logged in");
  }
}

class Student extends Person {
  study() {
    console.log(this.name + " is studying");
  }
}

class Teacher extends Person {
  teach() {
    console.log(this.name + " is teaching");
  }
}

const student = new Student("Ripon");
const teacher = new Teacher("Rahman");

student.login();
student.study();

teacher.login();
teacher.teach();
```

CONSOLE OUTPUT

```
Ripon logged in
Ripon is studying
Rahman logged in
Rahman is teaching
```

এখানে common functionality:

TEXT

```
login()
```

Person থেকে এসেছে।

আর specific functionality:

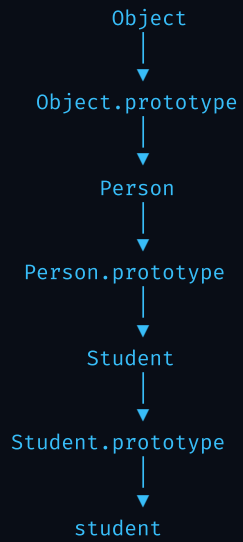
TEXT

```
study()
teach()
```

নিজ নিজ child class-এ আছে।

11.54 Prototype সম্পর্কে সবচেয়ে গুরুত্বপূর্ণ Concept

একবার এই diagram বুঝে ফেললে অনেক কিছু পরিষ্কার হয়ে যাবে:



আর actual instance lookup:

```
student
  ↓
Student.prototype
  ↓
Person.prototype
  ↓
Object.prototype
  ↓
null
```

MUST KNOW 🔥 Chapter 11 — Must Know

এই Chapter থেকে অবশ্যই জানতে হবে:

1. Prototype

JAVASCRIPT

RUN

```
Object.getPrototypeOf(obj);
```

2. Prototype Chain

```
object
↓
prototype
↓
prototype
↓
null
```

3. Own vs Inherited Property

JAVASCRIPT

RUN

```
obj.hasOwnProperty("name");
```

4. in

JAVASCRIPT

RUN

```
"name" in obj;
```

5. Object.create()

JAVASCRIPT

RUN

```
const child = Object.create(parent);
```

6. Constructor Function

JAVASCRIPT

RUN

```
function Student(name) {
  this.name = name;
}
```

7. new

JAVASCRIPT

RUN

```
const student = new Student("Ripon");
```

8. Constructor Prototype

JAVASCRIPT

RUN

```
Student.prototype
```

9. Prototype Method

JAVASCRIPT

RUN

```
Student.prototype.study = function() {};
```

10. instanceof

JAVASCRIPT

RUN

```
student instanceof Student;
```

11. Class

JAVASCRIPT

RUN

```
class Student {}
```

12. extends

JAVASCRIPT

RUN

```
class Student extends Person {}
```

13. `super`

JAVASCRIPT

RUN

```
super(name);
```

14. Method Overriding

JAVASCRIPT

RUN

```
speak() {}
```

Child class একই method redefine করতে পারে।

CHEAT SHEET

🗨 Quick Cheat Sheet

JAVASCRIPT

RUN

```
// Prototype দেখা
Object.getPrototypeOf(obj);

// Prototype সেট করা
Object.setPrototypeOf(obj, prototype);

// Prototype দিয়ে object তৈরি
Object.create(prototype);

// Own property
obj.hasOwnProperty("name");

// Own + inherited property
"name" in obj;

// Constructor
function Student(name) {
  this.name = name;
}

// Constructor prototype
Student.prototype;

// Prototype method
Student.prototype.study = function() {};

// Create instance
const s = new Student("Ripon");

// Instance check
s instanceof Student;

// Modern inheritance
class Student extends Person {}

// Parent constructor
super(name);

// Parent method
super.method();
```

নিজে হাতে এগুলো solve করার চেষ্টা করো।

Practice 1

একটি:

JAVASCRIPT

RUN

Person

constructor তৈরি করো।

Properties:

TEXT

name

age

Practice 2

`Person.prototype` -এ:

TEXT

introduce()

method তৈরি করো।

TEXT

My name is Ripon

Practice 3

`Student` নামে আরেকটি constructor তৈরি করো এবং `Person` থেকে inheritance করাও।

Student-এর property:

TEXT

department

Practice 4

Check করো:

JAVASCRIPT

RUN

student `instanceof` Student

student `instanceof` Person

student `instanceof` Object

তিনটির output কী হবে?

Practice 5

এই code-এর prototype chain বের করো:

JAVASCRIPT

RUN

```
class Animal {}  
  
class Dog extends Animal {}  
  
const dog = new Dog();
```

Expected structure:

```
dog  
↓  
?  
↓  
?  
↓  
?  
↓  
null
```

SUMMARY

🚀 Chapter 11 Final Summary — Object Prototype Model

আজকের সবচেয়ে গুরুত্বপূর্ণ ধারণা হলো:

```
JavaScript Object  
↓  
Prototype  
↓  
Prototype Chain  
↓  
Inheritance  
↓  
Constructor Function  
↓  
new  
↓  
Class  
↓  
extends  
↓  
super
```

সবচেয়ে গুরুত্বপূর্ণ mental model:

```
const student = new Student();
```

এরপর JavaScript কোনো method খুঁজলে roughly:

```
student  
↓  
Student.prototype  
↓  
Parent.prototype  
↓  
Object.prototype  
↓  
null
```

এই **Prototype Chain**-টাই JavaScript-এর object system-এর মূল ভিত্তি। `class` ব্যবহার করলেও ভিতরের ধারণাটি prototype-based।

NEXT CHAPTER

🚀 Chapter 12 Preview — Advanced Collections

Chapter 12-এ এরপর আমরা **Map, Set, WeakMap, WeakSet** — JavaScript-এর **Advanced Collections** শিখতে পারি।